Confidential and verifiable upstream inference over standard TLS 1.3

A no-trusted-hardware protocol for LLM routing nodes

Juraj Bednar <juraj@bednar.io>

Coauthors: Claude Opus 5.5, Claude Sonnet 5.5, Kimi K3,
DeepSeek V4.1 Flash, GPT-6 Astra, GLM-5.3 Flash

2026-10-10

Abstract

LLM routing nodes forward user requests to upstream providers with the node's own API key, so users must trust the node with their prompts, and the node must keep its key from its users. Enclave-based designs ask every user to verify an attestation of the node's software after every update. Our protocol rests instead on the provider's TLS certificate and on cryptography. The client is the TLS 1.3 client of the provider and the node relays the records. Client and node run the TLS key schedule as a two-party computation (share conversion over P-256, then a garbled circuit that only the node garbles and the client checks gate by gate with information-theoretic MACs): the node alone gets the key of the first application epoch, in which it writes its `Authorization` record and a KeyUpdate, and the client alone gets the keys for the body and the streamed response. Nobody holds the ECDHE or master secret, so the client cannot recover the API key even if it records the node's traffic. Zero-knowledge proofs anchored on hashes the computation outputs show the client that the node's record matches a published template, and show the node that the request ends in a public suffix pinning the model and token cap and that the billed usage is the provider's own report. Against a live provider on one host, a cold request reaches the provider after about 1.2 s and a pre-warmed one 5 ms after it is claimed, followed by a 0.10 s suffix proof. The price is about 22 MB from the node to the client and 1.2 MB back per request; on a home link with a 2 Mbit/s uplink the cold two-party computation takes about 7 s, and pre-warming moves it off the request path. We also describe a single-party variant that needs under a quarter of the bandwidth but protects the key only while no observer of the node's egress colludes with the client.

Contents

1. Introduction **2**
1.1 Contributions . . . 3

2. Model **3**

3. Preliminaries and the provider profile **4**
3.1 Provider profile . . . 4
3.2 TLS 1.3 key schedule . . . 4
3.3 Cryptographic building blocks . . . 5

4. Protocol **5**
4.1 Who holds which secret . . . 5
4.2 Epoch structure . . . 6
4.3 Setup, preprocessing and the joint key share . . . 6
4.4 The two-party key schedule . . . 7
4.5 Credential record (epoch 0) and π_{NK} . . . 8
4.6 Body (epoch 1) and π_{C2S} . . . 9

4.7 Response and usage disclosure (π_{C3S}) . . . 9
4.8 Settlement . . . 10
4.9 Aborts . . . 10

5. Security analysis **11**

6. Implementation and evaluation **13**
6.1 Prototype . . . 13
6.2 Component costs . . . 13
6.3 End to end . . . 14
6.4 Over a network and bandwidth . . . 16
6.5 Live verification . . . 17
6.6 The dual-execution version . . . 17

7. v1: the single-party variant **17**

8. Deployment pattern **18**
8.1 Denial-of-service protection and fees . . . 18

9. Related work and alternatives **19**

10. Conclusion **21**

Appendix A. Case study: the provider profile in practice **21**

References **22**

1. Introduction

Routing nodes and gateway services sit between users and upstream LLM APIs. A node holds an upstream account and forwards requests with its own API key; users pay the node (typically prepaid with e-cash tokens such as Cashu), and the node pays the upstream. Two trust problems follow.

The user trusts the node with everything. The node sees the prompt and the response; it can log, alter or replay them. This is a privacy problem (prompts contain source code, contracts, medical text) and a correctness problem (the node can substitute models or parameters).

The node must keep its credential from the user, and both must agree on the bill. The node's upstream API key must never leak to users. Charging must be defensible: the user should be able to check that the tokens billed are the tokens the upstream actually produced and reported.

The usual answer to the first problem is confidential computing: run the node, or the model, inside a trusted execution environment (TEE) and let users verify a remote attestation. That moves trust from the operator to the hardware vendor and to the measured software. For the guarantee to mean anything, a user has to know which software measurement is acceptable, which requires a reproducible build and a review of the whole stack that touches the plaintext, and has to repeat that after every update. In practice few users do this, and the attestation becomes a badge rather than a check.

This paper takes a narrower route. A user's request to a provider is already protected by TLS, and the provider is already authenticated by a WebPKI certificate. We **hand the TLS client role to the user and let the node relay ciphertext**. The node contributes its credential to the request without the user learning it, and cryptographic proofs pin the few bytes each side may write and bind billing to the provider's own numbers. There is no attested software to track: the user checks a certificate and a handful of proofs whose statements are fixed by the protocol.

The difficulty is the credential. If the user's client runs the TLS handshake alone, it knows every traffic secret, including the key of the record that carries the node's API key; the key is then hidden only as long as the client never sees that record's ciphertext. We remove that condition by computing the handshake key schedule jointly: the node obtains the key for its own record, the client obtains the keys for the body and the response, and neither can derive the other's. The

two-party computation (2PC) covers only the key schedule, a fixed workload of 31 SHA-256 compressions, never the payload.

1.1 Contributions

1. A TLS 1.3 protocol (section 4) in which the first application epoch belongs to the relay node, which writes its credential record and a KeyUpdate under a key only it holds, while the body and the response run under keys only the client holds. The handshake key schedule runs as a two-party computation secure against a malicious peer, in which only the node garbles and the client checks every gate against masks the node has authenticated to it. Neither party ever reveals its inputs; a cheating node can learn at most $\log_2 5 \approx 2.3$ bits per session about values used only in that session, and the client uploads 1.2 MB (section 4.4).
2. A request-integrity construction: the client proves that its request body ends in an agreed public JSON suffix that pins every cost-relevant parameter, and the provider sees the complete request only after the proof verifies (section 4.6). The node proves that its hidden record matches a published template (section 4.5).
3. A verifiable-accounting construction: the client proves that the upstream's terminal usage event is authentic decrypted output of the session, anchored at the end of the relayed transcript, and the node charges exactly that usage at the rates of the offer the client verified, through its ordinary billing (sections 4.7, 4.8). Proofs that decrypt reveal one bit (equal or not), never decrypted bytes.
4. A **provider profile** (section 3.1) that separates what any upstream must satisfy from provider-specific details.
5. A working implementation verified end to end against a live provider, with measurements of every component and of the whole path, cold and pre-warmed (section 6), a comparison with an earlier dual-execution version of the 2PC (section 6.6), and a comparison with a simpler single-party variant that trades the credential guarantee for bandwidth (section 7).

2. Model

Parties:

- **C, client** (a user's daemon or app). TLS 1.3 client of the upstream session; the only holder of the body and response traffic secrets; payer of the node.
- **N, node.** Holds the upstream API key; owns the TCP connection to the upstream and relays it; the only holder of the first application epoch's key; bills C through its ordinary accounts.
- **U, upstream** (any provider whose profile, section 3.1, is satisfied). Unmodified.

Channels: C to N is a control channel, a byte relay and one channel per proof or 2PC (in our deployment, WebSockets); N to U is ordinary TCP:443.

Goals:

ID	Property
P1	Credential confidentiality: only N knows the upstream API key. C learns nothing about it even if it sees every byte N sends to U.
P2	Content confidentiality: only C (and U) can read the prompt and the response. N sees ciphertext plus the public billing data: the usage event, or on an upstream error the response head.
P3	Integrity: C can verify that the request U received is C's own content behind the agreed public template content (not its exact wire bytes, residual risk 1), and that the response is exactly what U sent (tool calls included).
P4	Verifiable accounting: the charge is computed from the token usage the upstream itself reported in the session, verified by N (who settles) and by C (who checks the signed receipt against its own plaintext).
P5	Upstream authenticity: both parties verify that the session terminates at a server holding a valid certificate for the upstream hostname.
P6	Freshness: every session is live and bound to one payment; nothing can be reused across sessions.

Non-goals: hiding the prompt from U (U processes it by definition); proving which model weights ran; hiding traffic

volume and timing from N (section 5); and third-party verifiability. The proofs convince the two parties of the session, which is what settlement needs. They are interactive and designated-verifier, and C, holding the server's application key, could forge server records for anyone but N. A receipt is N's signed statement, not a proof to outsiders.

Adversaries: either C or N may deviate arbitrarily from the protocol (they are malicious towards each other); U follows TLS and its own API, and is trusted with the prompt and the API key as it is today. Assumptions: the standard hardness of ECDH over P-256, SHA-256, HMAC and AES-GCM; fixed-key AES as a correlation-robust hash (the usual assumption of free-XOR garbling); security of oblivious transfer and correlated-OT extension (KOS, Ferret) against a malicious party, and of VOLE-based zero knowledge; the WebPKI. No trusted hardware.

3. Preliminaries and the provider profile

3.1 Provider profile

The protocol is defined for any upstream satisfying:

1. **TLS 1.3** with an AES-GCM suite in the SHA-256 family (this work: `TLS_AES_128_GCM_SHA256`) and ECDHE over **secp256r1**. No compression, no 0-RTT, no HelloRetryRequest for a P-256 share.
2. **HTTP/1.1** with explicit `Connection: close` and `Accept-Encoding: identity` (record-level epoch interleaving requires HTTP/1.1 framing; HTTP/2 would need a different record mapping).
3. **KeyUpdate accepted mid-request** (client-initiated, `update_not_requested`).
4. **No processing before the complete body.** The provider starts work only once the whole `Content-Length` body has arrived. N relies on this to stop a request whose suffix does not verify (section 4.6).
5. **Parameter pinning semantics**: a JSON request whose *effective* parameters are fully determined by a public suffix that the client can append. The provider must:
   - keep the last value of duplicate top-level keys;
   - let a later object value replace an earlier one rather than merge with it;
   - reject malformed JSON and unknown fields, so the private prefix cannot leave an object or a string open for the suffix to land in.
6. **Cost-relevant parameters overridable by the suffix**: model, token caps (which must cap reasoning tokens too), sampling count, and provider feature flags that change price, including any injected system prompt. The provider must reject any other price-changing field. The size of the prompt needs no bound, because the node reserves for a full context window (section 4.6).
7. **A streaming (or terminal) usage event** carrying input/output token counts and, ideally, a provider-computed cost figure, emitted before the response ends.
8. **No reflection of the request head into the response.** C reads the response, so a provider that echoed the `Authorization` header (for example in an error body) would hand C the key.

Items (5) and (6) quantify over every field and input, which no finite test can establish. For the case-study provider (Appendix A) a live regression gate tests items 1 to 7 with sampled cases; the requirements hold as far as those samples show, and the gate is re-run after provider changes. Item 8 is a property of the provider's error handling that the gate does not test. Everything except the specifics of items 4 to 7 is provider-generic: the epoch structure, the 2PC, the proofs and the record assembly do not change per provider; only the offer (template, pinned keys, price list, usage parser) does.

3.2 TLS 1.3 key schedule

The TLS 1.3 key schedule (RFC 8446 section 7.1, SHA-256) derives, from the ECDHE shared secret Z and the transcript hashes th_SH (through ServerHello) and th_SF (through server Finished):

$$HS = \text{HKDF-Extract}(\text{Derive-Secret}(ES, \text{"derived"}), Z)$$

$$c_{hs}, s_{hs} = \text{HKDF-Expand-Label}(HS, \text{"c hs traffic" / "s hs traffic"}, th_{SH})$$

$$MS = \text{HKDF-Extract}(\text{Derive-Secret}(HS, \text{"derived"}), 0^{256})$$

$$c_{ap,0}, s_{ap,0} = \text{HKDF-Expand-Label}(MS, \text{"c ap traffic" / "s ap traffic"}, th_{SF})$$
$$c_{ap,i+1} = \text{HKDF-Expand-Label}(c_{ap,i}, \text{"traffic upd"})$$

where ES is a public constant without a PSK. Each traffic secret yields an AES key K and IV through the "key" and "iv" labels. A record with sequence number n uses nonce $IV \oplus n$; its ciphertext is the plaintext XOR the AES-CTR keystream from counter 2, and its tag is $\text{GHASH}_H(AAD, C) \oplus \text{AES}_K(J_0)$ with $H = \text{AES}_K(0^{128})$. Every step is a one-way HMAC-SHA-256, so any set of secrets on the right of a derivation reveals nothing about the secrets that are not derived from it.

KeyUpdate: a sender can rotate its own sending keys at any record boundary ($c_{ap,i} \to c_{ap,i+1}$); the receiver updates likewise. Sequence numbers restart in each epoch.

3.3 Cryptographic building blocks

VOLE-based zero knowledge. We use the interactive QuickSilver-style ZK of TLSNotary's `mpz2`, where SHA-256 (as a compression-function circuit) and AES-128 are available as gadgets. Both parties are online during the session, so interactivity costs only round trips. Every plaintext comparison in our statements is done inside the circuit and only a single equality bit is decoded.

Garbled circuits. A garbler encodes a Boolean circuit so that an evaluator who holds one label per input wire learns the output and nothing else. With free XOR, the two labels of every wire differ by a global offset Δ that only the garbler knows, and XOR gates cost nothing. With point-and-permute, the last bit of the label the evaluator holds is a pointer bit $\Lambda_w = z_w \oplus \lambda_w$: the wire's true value z_w hidden by a mask λ_w that the garbler chose. Half-gates garbling (Zahur, Rosulek and Evans 2015) sends 256 bits per AND gate; three-halves garbling (Rosulek and Roy 2021) splits labels into halves and sends $1.5\kappa + 5 = 197$ bits per AND gate. Plain garbling protects the garbler's inputs from the evaluator, but nothing stops a malicious garbler from garbling a different circuit.

IT-MACs and authenticated garbling. An information-theoretic MAC on a bit x held by one party gives that party x and $M = K \oplus x\Delta_B$ over $GF(2^{128})$, and gives the other party the key K and a global key Δ_B. The holder can open x only to its true value unless it guesses Δ_B, and MACs are linear, so XORs of authenticated bits are authenticated for free. Random correlated oblivious transfer (KOS, or Ferret on top of it) produces such MACs cheaply, and a QuickSilver-style check verifies a whole batch of products $t = x_i x_j$ at the cost of two field elements. *Authenticated garbling* (Wang, Ranellucci and Katz 2017, and its successors) uses IT-MACs on the wire masks to make a single garbled circuit secure against a malicious garbler. In the full constructions the masks are secret-shared between the two parties, which removes all leakage but needs authenticated AND triples whose generation costs more bandwidth than garbling twice. We use a garbler-only form (section 4.4): the garbler keeps the masks, authenticates them to the evaluator, and the evaluator checks every AND gate with one batched check. The price is a few bits of selective-failure leakage per session.

Share conversion (ECtF). Two parties holding ECDH contributions can convert the two halves of a point sum into additive shares of its x-coordinate with a few multiplicative-to-additive (M2A) and additive-to-multiplicative (A2M) conversions over the base field, built on oblivious linear evaluation. TLSNotary uses the same construction for its key exchange.

4. Protocol

4.1 Who holds which secret

The protocol is built around a split of the TLS secrets. C and N jointly emulate one TLS client; the 2PC releases to each party only what it needs:

Value	C	N	Purpose
ECDHE scalar, Z	additive share	additive share	key exchange
HS, MS, $c_{ap,0}$	never	never	inside the 2PC only

Value	C	N	Purpose
c_{hs}, s_{hs}	yes	yes	both verify the handshake (P5)
K_0, IV_0 (from $c_{ap,0}$)	**never**	yes	N's credential record (P1)
$c_{ap,1}$	yes	**never**	C's request body (P2)
$s_{ap,0}$	yes	**never**	the response (P2)
$h_1 = \text{SHA256}(c_{ap,1})$, $h_2 = \text{SHA256}(s_{ap,0})$, $h_k = \text{SHA256}(K_0 \| IV_0)$	yes	yes	public anchors for the proofs

$c_{ap,1}$ is derived from $c_{ap,0}$, but not the other way round, so C's body key does not reveal N's record key. Neither K_0 nor IV_0 yields $c_{ap,0}$. U sees an ordinary TLS client.

4.2 Epoch structure

Epoch	Key	Writer	Content
Handshake	c_{hs} / s_{hs}	C (U)	ClientHello with the joint key share; client Finished computed by C from c_{hs}
0	$c_{ap,0}$ (K_0, IV_0)	**N**	sequence 0: the credential record (request head with `Authorization`); sequence 1: `KeyUpdate(update_not_requested)`
1	$c_{ap,1}$	C	the JSON request body; its final record is the public suffix, released to U by N only after π_{C2S}
Response	$s_{ap,0}$	U	streamed response; N relays ciphertext

C writes nothing in epoch 0 and starts epoch 1 at sequence 0. N writes exactly two records, atomically. U processes epoch 0 as an ordinary client's first records followed by a KeyUpdate, and decrypts everything after it under $c_{ap,1}$.

4.3 Setup, preprocessing and the joint key share

Setup. C opens a control channel to N and sends its ordinary node credential, the signature of the offer it verified, the model, the token cap and the body length. N validates the credential, freezes the offer's rates, reserves funds for the request (section 4.6) and only then dials its relay process. N opens no connection to U before a valid setup.

Preprocessing. Before any TLS byte is sent, N garbles both phases of the key-schedule circuit (section 4.4) and streams the tables to C, the parties authenticate N's wire masks to C and run the checks on them, and they run the oblivious-transfer setup for C's input bits and the random OLEs for the share conversion. This step is independent of all inputs. It moves about 22 MB from N to C and 1.2 MB from C to N, and it cannot be reused: a garbled circuit is single-use, and each session needs one handshake and one credential record. Section 6 discusses how pre-warming moves it off the request path.

Joint key share. C draws a scalar a_C, sends a commitment $\text{SHA256}(P_C \| \rho)$ to $P_C = a_C G$, receives $P_N = a_N G$ from N and opens. Both compute $P = P_C + P_N$. Committing first prevents C from choosing $P_C = T - P_N$ for a point T whose discrete logarithm it knows. Scalars are fresh in every session. The key exchange uses P-256 rather than X25519 because the joint share needs full point addition, which X25519's x-only Montgomery form does not offer.

ClientHello enforcement. C builds a ClientHello with exactly one key share, the joint point P, on secp256r1; `supported_groups` lists only secp256r1; one cipher suite; no PSK and no early data. **N parses the ClientHello and relays it only if all of this holds.** Without this check C could add a share of its own, finish the handshake alone and learn every secret. N rejects a HelloRetryRequest and checks that the ServerHello selects secp256r1 and the offered suite, then extracts U's share S.

4.4 The two-party key schedule

ECtF. C computes $a_C S$ and N computes $a_N S$. A point-addition share conversion (two A2M and one M2A over the P-256 base field) turns these into additive shares $x_C + x_N = x(a_C S + a_N S) \bmod p$ of the ECDHE secret Z, without either party learning Z.

Phase A (after the ServerHello). A garbled circuit takes x_C, x_N, th_SH and both parties' random masks. It adds the shares modulo p (about 770 AND gates), computes HS, its keyed HMAC states, c_{hs}, s_{hs}, the "derived" secret and MS. It outputs c_{hs} and s_{hs} in the clear and MS only as $MS \oplus r_C \oplus r_N$, so that afterwards C's share of MS is r_C and N's is $MS \oplus r_C$. Neither share says anything about MS. Phase A must finish before U gives up waiting for the client Finished; locally it takes about 33 ms, check included.

Handshake verification. With s_{hs}, C decrypts U's encrypted handshake messages in its own TLS stack and verifies the certificate chain to the WebPKI roots, the hostname, CertificateVerify and the server Finished, then computes and sends its Finished from c_{hs}. N independently decrypts the same messages, reconstructs the transcript and verifies the chain, hostname, CertificateVerify and both Finished messages, fail-closed (P5). A wrong Z, for example after a deviation in ECtF, fails the server Finished here.

Phase B (after the server Finished). A second circuit takes the MS shares, th_SF and fresh masks, and computes $c_{ap,0}$, $s_{ap,0}$, $c_{ap,1}$, the record key and IV of $c_{ap,0}$, and the three anchors. It outputs $(s_{ap,0} \| c_{ap,1}) \oplus m_C$, which only C can unmask, $(K_0 \| IV_0) \oplus m_N$, which only N can unmask, and h_1, h_2, h_k in the clear.

Every HKDF-Expand-Label in the schedule fits one inner block, and keyed HMAC states are computed once per secret and reused, so phase A costs 14 SHA-256 compressions (298,088 AND gates) and phase B 17 (358,091 AND gates).

Garbling and authenticated masks. Only N garbles. In preprocessing, N garbles each phase once with three-halves garbling under its global offset Δ_A, so a session garbles 656,179 AND gates. The mask λ_w of every wire is the pointer bit of N's label for the value 0, and C only ever sees the pointer bits $\Lambda_w = z_w \oplus \lambda_w$. C draws a global MAC key Δ_B (128 bits, 127 of them random). Using random correlated OT in which C is the sender (Ferret, bootstrapped by KOS), N authenticates to C the mask of every circuit input and every AND output (the *base* wires) and, for every AND gate k with inputs i, j, the product $t_k = \lambda_i \lambda_j$: about 1.3 million IT-MACs. The masks of all other wires follow by linearity. N sends its tables and these commitments first; only then does C send a 32-byte seed, from which both derive a challenge χ and a per-session salt. N then proves all products t_k with one QuickSilver-style check, opens the masks of C's input wires, of the transcript-hash wires and of every output wire (the bits and one hash of their MACs), and sends one check row per base wire and a value Y_N.

Evaluation and the consistency check. Online, C obtains the labels of its inputs by oblivious transfer, and N sends the labels of its own inputs and of the transcript hash. C checks the pointer bits of its own inputs and of the transcript hash against the opened masks, so N can neither substitute C's inputs nor feed in a different transcript. C evaluates and then checks all AND gates at once. Gate $k = (i, j \to o)$ was evaluated consistently with the committed masks if and only if

$$e_k = \lambda_o \oplus t_k \oplus \Lambda_i \lambda_j \oplus \Lambda_j \lambda_i \oplus \Lambda_o \oplus \Lambda_i \Lambda_j = 0.$$

Under the MACs, $\sum_k \chi^k e_k = 0$ becomes a linear equation between Y_N and the check rows on one side and, on the other, a value C computes from its MAC keys and its active labels. N does not know the pointer bits, so each row is padded with hashes of the wire's two labels and only the label C actually holds opens it. C accepts if and only if the two sides agree. The rows and Y_N are sent truncated to 64 bits, while all arithmetic stays in $GF(2^{128})$. The hash in each row is tweaked by the per-session salt, which N learns only after it has committed its tables, its offset and its keys (section 5 explains why this matters).

Output. C decodes its outputs with the authenticated masks. Only then does it send N the output pointer bits and a hash of its output labels. N recomputes those labels from its own keys, checks the hash and decodes its outputs.

Private outputs are masked inside the circuit with a mask only their owner inputs, so each party decodes only a one-time-padded value of the other's output. C cannot change N's outputs, because that needs an inactive output label, and inactive labels need Δ_A. On any failure both parties abort. **No output is sent to U, used to encrypt, or revealed before these checks pass.** Online, each phase takes about 1.5 round trips and 35 to 45 ms of computation.

What a cheating party gains. A cheating N can garble a different circuit, lie about a mask, a product or an opened bit, corrupt check rows, or run the oblivious transfer for C's inputs with a wrong correlation. Each of these either fails the product check, the openings, the pointer checks or the consistency check, or leaves C's outputs correct (section 5). What N can still do is make an abort depend on C's private values (selective failure). In each phase it observes one of three outcomes (C aborted, C's output labels were rejected, success), and the session ends at the first failure, so there are at most five outcomes per session: N learns at most $\log_2 5 \approx 2.3$ bits. A cheating C, the evaluator, sees the tables, its active labels, uniformly random pointer bits, the opened masks of values it already knows or decodes (its inputs, the public transcript hash, its outputs), check rows padded by hashes of labels it does not hold, and Y_N, which it can compute itself. It learns nothing beyond its outputs and can only choose its own inputs, as in any 2PC. A party that feeds different MS shares or masks to phase B than phase A gave it only obtains keys nobody knows, which U rejects. ECtF uses semi-honest conversions; a deviation there only shifts Z, which the server Finished check catches, or causes a selective failure that is already counted above.

Why the leakage cannot accumulate. The bits N can learn are about C's inputs to that session: its ECDHE scalar share, its MS share and its masks. The protocol keeps one invariant: **no 2PC input and no garbling key is ever reused across sessions.** Scalars, shares, masks, labels, Δ_A, Δ_B, the MAC keys and the salt are all drawn fresh for each session and used once, and an aborted session is discarded. Bits leaked in different sessions are therefore about independent values and never add up to any secret. The API key is never an input to the 2PC: it exists only in N's epoch-0 record.

Inputs are never opened. Every private input enters only as a garbled input or an OLE input, and nothing is revealed after the session. This distinguishes the construction from 2PC-TLS designs in which one party opens its shares afterwards (section 9), which N cannot do here: its shares would give C the master secret and therefore K_0.

What each party learns, in total:

Party	Learns	Does not learn
C	c_{hs}, s_{hs}, $s_{ap,0}$, $c_{ap,1}$, h_1, h_2, h_k; whether a check passed	Z, HS, MS, $c_{ap,0}$, K_0, IV_0, the API key
N	c_{hs}, s_{hs}, K_0, IV_0, h_1, h_2, h_k; whether a check passed; the suffix and the usage event (as one-bit-checked public values)	Z, HS, MS, $s_{ap,0}$, $c_{ap,1}$, prompt, response
U	an ordinary TLS 1.3 session	that two parties are involved

4.5 Credential record (epoch 0) and π_{NK}

After phase B and the handshake verification, N fills the published head template with its API key and C's declared body length, encrypts it under K_0 at sequence 0, and seals `KeyUpdate` at sequence 1. It sends C a commitment $c = \text{SHA256}(header \| C_1)$ to the ciphertext and proves its content.

π_{NK} (N to C).

- *private*: K_0, IV_0, the key bytes;
- *public*: h_k, the template with the key positions, the key alphabet, $|head|$, the record header, the commitment c;
- *statement*: $\text{SHA256}(K_0 \| IV_0) = h_k$; the AES-CTR keystream under K_0 with nonce $IV_0 \oplus 0$; $C_1 = (head \| \texttt{0x17}) \oplus$ keystream; $\text{SHA256}(header \| C_1) = c$; $head$ equals the template outside the key positions; every key byte is in the alphabet. Only the digests and the alphabet bit are decoded.

C rebuilds every public input from the signed offer, its own body length and the h_k that its own 2PC output, so N cannot prove a different template or a key other than the one the 2PC produced. The proof binds the head's content

and length. It does not bind the bytes N actually writes: C never sees C_1, so a different head of the same length is not detectable by C (section 5, residual risk 1).

When π_{NK} has verified, C claims the session. N writes the credential record and the KeyUpdate in one write; C's epoch-1 records, held by N until then, follow. There is no client proof in this step: N never handed out anything that C must prove correct.

4.6 Body (epoch 1) and π_{C2S}

C writes body = body_private ‖ suffix under $c_{ap,1}$ from sequence 0, with $|body|$ fixed at setup. body_private opens the JSON object and may carry messages, tools and other schema-valid fields. The suffix is the public string that pins every cost-relevant parameter: model, token caps, `n`, streaming, usage reporting and provider feature flags. N's relay builds it from the model and token cap declared at setup and accepts no other. The provider resolves the effective values to the suffix (section 3.1, item 5).

C sends the suffix as its own final record. N forwards the earlier body records but **holds the record that completes the body** until π_{C2S} verifies. N enforces that the epoch-1 plaintext is exactly $|body|$ bytes, so that record is identifiable. Until it is released U has an incomplete request and does nothing (section 3.1, item 4). If the proof fails, N closes the connection: no inference ran and nothing is billed. A client therefore cannot run a different model or a larger cap than N reserved for.

π_{C2S} **(C to N, before U can start).**

- *private*: $c_{ap,1}$;
- *public*: h_1, the record's sequence number, a ciphertext window of the final record and its offsets, the suffix;
- *statement*: SHA256$(c_{ap,1}) = h_1$; derive K_1, IV_1; CTR-decrypt the window; its end equals the suffix. The verifier decodes one bit.

Because the anchor is a hash of the secret that the 2PC itself output to both parties, the statement does not recompute the key schedule: it costs 7 SHA-256 compressions and 15 AES blocks for the 230-byte suffix, 0.16 s locally.

Parameter validation by C. The public parameters of π_{C2S} and π_{C3S} come from N. C checks each against its own view before proving: π_{C2S} must be a window ending at the end of C's own final body record (sequence number, offset and ciphertext bytes as C sent them) with C's suffix as the expected value; π_{C3S} (section 4.7) must cover the final server records exactly as C received them, matched from the end of the transcript with at most one trailing close_notify record, with C's own disclosure as the expected value; the anchors must equal C's own 2PC outputs. N therefore cannot aim a proof, or its one-bit result, at other plaintext.

Reservation. The prompt is opaque to N, so the reservation never discounts it: it is $\max(m, \lceil (L \cdot r_{in} + T \cdot r_{out})/1000 \rceil)$ msat, where L is the model's prompt limit (context length minus its completion limit), T the pinned token cap, r_{in}, r_{out} the per-1k rates of the frozen offer and m the node's minimum charge. For the case study (`deepseek-v4-flash`, L = 104,858, T = 40) this reserves 2,282 msat; the session cost 63.

4.7 Response and usage disclosure (π_{C3S})

U's response streams under $s_{ap,0}$; C decrypts it and verifies every GCM tag natively (P3 for the response) and streams tokens to the application live. At close, N signs a **session attestation** binding the session id, the certificate chain hash, the two handshake transcript hashes (through ServerHello and through the server Finished), the record counts in each direction, the SHA-256 of all bytes sent to U and of all bytes received from it, the commitment to the credential record (c_{commit}) and the time. N's relay signs it with a per-process Ed25519 key and N signs it again with its Nostr key; C checks N's signature and that the session id and both transcript hashes are its own.

C then discloses R: the plaintext from the boundary of the last usage-bearing data event through the stream terminator, with its chunk framing, anchored at the end of the transcript N relayed.

π_{C3S} **(C to N).**

- *private*: $s_{ap,0}$;

- *public*: h_2, the final records' ciphertexts and sequence numbers, R, and a count `skip` of undisclosed bytes after R;
- *statement*: SHA256$(s_{ap,0}) = h_2$; derive the server key and IV; CTR-decrypt the covered records; R ends `skip` bytes before the end of the response; in-circuit equality with R, decoded as one bit. For a usage disclosure `skip` is 0.

The choice of R is unambiguous: JSON strings cannot contain raw CR/LF, so event boundaries and HTTP chunk framing cannot be forged from model output. We checked this by prompting the model to emit fake usage events; every imitation stayed escaped inside a string (Appendix A).

4.8 Settlement

Confidential sessions use the node's ordinary billing; nothing about money is specific to the protocol:

1. **Setup.** C authenticates with the credential it already uses for normal requests, either an API key or a one-off e-cash token that the node turns into one, and names the offer it verified by its signature. N keeps an offer it signed for ten minutes after it last served it (at most 512 offers) and freezes that offer's per-model rates for the session; an unknown or expired offer is refused (HTTP 409) and C refetches it. N reserves on the credential (section 4.6) before any connection to U is opened.
2. **Charge.** N's relay parses R (dechunk, take the usage event and the terminal marker) and requires the event's model to equal the setup model; N requires it again before charging. N charges exactly the usage at the frozen rates, $\lceil (r_{in}(p-c) + r_{cached}\min(c,p) + r_{out}\cdot o)/1000 \rceil$ msat for p prompt tokens of which c cached and o completion tokens, capped at the reservation, through the node's ordinary finalize path.
3. **Receipt.** N signs (BIP-340 Schnorr, over canonical JSON) a receipt with the session id, model, usage, cost, reservation, the resulting balance, the frozen rates, its key, the SHA-256 of the exact attestation bytes, and the time. C requires the signature, takes every field from the signed bytes, and checks the session id and attestation hash against its own session, the usage against the plaintext it already has, and the cost against that usage at the signed rates. Unused balance stays on the credential and is withdrawn through the node's normal refund endpoint.

If C does not disclose after U received the request, N charges the full reservation, which is why disclosure is always in C's interest.

Upstream errors. An HTTP-level error from U (for example 503, 429, or a 400 for a request U rejects) carries no usage event. C then discloses only the response *head*: the status line, the header lines and the blank line that ends them. π_{C3S} proves it with `skip` set to the length of the rest of the response, so the head is anchored at a known distance from the end of the transcript and the error body, which can quote the request, stays private. For statuses the provider answers before doing any work (4xx, 503), N releases the reservation; other 5xx are charged like a missing disclosure. The head is unambiguous for the same reason as the usage event: a strict status line followed by well-formed `Name: value` header lines cannot come from model output, nor from chunk framing.

Pricing the node's work (optional). A session costs N about 22 MB of upload and 0.5 s of CPU before U sees a byte, and a pre-warmed session (section 6) costs it before any request exists, so an unpriced armed session is a cheap way to make N work. N can therefore publish, in the same signed offer, a rate surcharge (already folded into the signed price list, so the check above is unchanged), a session fee f_s and an abandon fee $f_a \geq f_s$. C accepts both fees by echoing them in its setup; N refuses a setup that does not, so a client that does not know about fees is never charged one. N reserves the amount above plus $\max(f_s, f_a)$. A completed session is charged the usage at the signed rates plus f_s (capped at the reservation), and C's receipt check becomes $cost \leq \lceil (r_{in}p + r_{out}o)/1000 \rceil + f_s$. A 4xx/503 from U is charged f_s: the request reached U and N did the 2PC. A session that N's relay accepted but that ends without a complete request reaching U (armed and never used, aborted, C gone) is charged f_a instead of nothing. N cannot tell a pre-warmed session from a cold one and does not need to. With the per-address limit on unused sessions this bounds what an idle or hostile client can make N do: each unused session costs its holder f_a. Both fees are 0 by default, and then the protocol is exactly as described above.

4.9 Aborts

Situation	Outcome
Any 2PC check fails, the ClientHello is rejected, or the handshake does not verify	session discarded before anything is written under K_0; reservation released (less the abandon fee f_a, if N charges one)
Abort before N releases the final body record (including a failed π_{C2S}, or an armed session never used)	U never received a complete request; reservation released (less f_a)
Usage disclosed	charged per section 4.8
U returned a 4xx or 503 (disclosed)	reservation released (less the session fee f_s)
Released, no usage disclosed (or another 5xx)	full reservation charged; C keeps its transcript and attestation

5. Security analysis

P1 (credential). The API key exists in the protocol only as plaintext of N's epoch-0 record, encrypted under K_0. C never obtains K_0, IV_0, or any secret from which they can be derived (Z, HS, MS, $c_{ap,0}$): the 2PC outputs these to nobody or to N alone, masked under N's mask, and nothing is opened afterwards. The secrets C does receive (c_{hs}, s_{hs}, $s_{ap,0}$, $c_{ap,1}$) are one-way HKDF outputs that are not ancestors of K_0, and h_k is a hash of a 28-byte uniformly random value. So a C that records every byte N sends to U, alone or with an observer on N's network path, holds AES-GCM ciphertext under a key it does not know. A cheating C learns nothing from the 2PC beyond its own outputs (section 4.4), and the API key is not a 2PC input at all. C cannot cut N out of the key exchange: N relays only a ClientHello whose single key share is the joint point. The commitment to C_1 that N sends is also safe: without K_0 it does not allow testing guesses of the key.

P1 still assumes that U does not send the key back to C (profile item 8), and that N's own process and U are not compromised.

P2 (content). The body runs under $c_{ap,1}$ and the response under $s_{ap,0}$, which the 2PC outputs only to C, masked with C's mask. N's outputs (c_{hs}, s_{hs}, K_0, IV_0, the anchors) are not ancestors of either. A cheating N that garbles a wrong circuit, for example one that outputs C's keys in a form N can read, is caught by C's consistency check before C uses any key, except with probability about 2^{-62} per session (below), and learns at most $\log_2 5 \approx 2.3$ bits about C's fresh, single-use inputs. The proofs that decrypt under C's keys (π_{C2S}, π_{C3S}) output one equality bit, on parameters C has checked against its own records, so N cannot use them as a decryption oracle. What N learns in plaintext is public by design: the suffix, the usage event, or on an error the response head.

Soundness of the 2PC against a cheating N. N fixes every base mask and every product t_k under Δ_B before it sees χ, and learns nothing about Δ_B (security of the correlated OT). The product check fails for a wrong t_k except with probability $2n/2^{128}$ for n gates. If some AND gate was evaluated inconsistently, then $E = \sum_k \chi^k e_k \neq 0$ except with probability $n/2^{128}$ over χ, and C then accepts only if the 64-bit projection of $\Delta_B E$ equals a value N fixed without any information about Δ_B. For $E \neq 0$ the map $\Delta \mapsto \pi(\Delta E)$ is onto $GF(2)^{64}$, so with 127 uniform bits in Δ_B this happens with probability at most 2^{-63} per phase, about 2^{-62} per session. N cannot grind: it learns nothing about Δ_B, and the first failure ends the session. Once every gate is consistent, C's outputs are the correct function of its own inputs, N's effective inputs and the real transcript, because C's inputs and the transcript hash are pinned by the pointer checks and C decodes with authenticated masks.

The salt. Truncating the rows to 64 bits needs one more step. With $E = 0$, a label that is not N's honest label but has the right pointer bit changes no output; it only changes the check sum by the difference of two truncated hashes. Without a salt, a node that reused its labels across sessions could precompute once, with about 2^{64} work, a table deviation whose 64-bit hash difference is zero. That deviation would pass the check and, through C's output-label hash, tell N which of two evaluation paths C took: leakage without an abort. Because the row hash is keyed by a salt from C's seed, and everything that fixes such a label (tables, Δ_A, N's keys) is committed before the salt is known, the hash difference is a fresh 64-bit value and the check fails except with probability 2^{-64}. Labels N chooses online (its own inputs, the transcript hash, the corrections for C's inputs) shift by amounts N already knows, so they carry no information about C. An honest node never reuses labels; the salt keeps the check sound against one that does.

P3 (integrity). C's body records are authenticated under a key only C has. N holds K_0 and can write any epoch-0 records (below), but none of them carries C's data. The suffix is proven to be the body's tail, so the effective parameters are exactly the agreed ones; response tags are verified by C.

P4 (accounting). The billed usage is a substring of the authenticated response, proven under the session's server secret and anchored to the end of the transcript. The anchors h_1, h_2 are outputs of the same 2PC that produced the keys, and N accepts them only through C's authenticated output report, so C cannot prove against a secret of its choosing without a SHA-256 preimage. N computes the suffix from the setup values, accepts only that suffix, and lets U see the complete request only after π_{C2S} has verified, so U runs exactly the reserved model and cap. The receipt binds the result to the session id and the reservation.

P5 (authenticity). Both parties verify the certificate chain, hostname, CertificateVerify and both Finished messages: C in its TLS stack, N from the handshake secrets that the 2PC gives it in the clear.

P6 (freshness). The session id is a hash of fresh nonces and the node's one-time reservation id; the TLS randoms, the ECDHE scalars, the masks, the garbled tables, both global offsets, the MAC keys and the salt are fresh per session.

Residual risks.

1. *N controls epoch 0.* N holds K_0, so it can write any number of records in epoch 0, not only the proven head, and π_{NK} binds the head's content, not the bytes N writes. N could, for example, use another account or endpoint, or add a header that enables provider-side retention. C cannot see N's wire bytes and cannot detect this. The effect is bounded: epoch 0 carries none of C's data, C's own records stay authenticated under $c_{ap,1}$, the response is readable only by C, and a broken request shows up as a failed response. A node could always choose to misuse its own account; what the protocol guarantees is that it cannot read or alter C's content.
2. *The 2PC is a new, unpublished variant.* The garbler-only authenticated garbling, its consistency check with garbler-held masks, the 64-bit projection and the salt are argued in section 4.4 and above, but this variant has no published proof, and the construction as a whole needs external review. The three-halves garbler is our own implementation: it was rebuilt from the scheme's structure, with the evaluator's matrices taken from a public reimplementation, and it is checked exhaustively for correctness. Correctness does not depend on those matrices but the privacy argument does, so they must be checked against Rosulek and Roy's paper. The MAC layer assumes that `mpz2`'s Ferret and KOS are secure against a malicious receiver (here N). ECtF is semi-honest, as in TLSNotary. The dual-execution version (section 6.6) rests on a simpler, better-studied construction, and the single-party variant (section 7) avoids the 2PC altogether.
3. *Selective failure.* Each 2PC session can leak at most $\log_2 5 \approx 2.3$ bits about the client's ECDHE share, MS share and masks through the abort pattern. No 2PC input or garbling key is reused across sessions, so the bits never accumulate on any secret, and the API key is not a 2PC input. A node that aborts sessions on purpose is visible to its clients.
4. *P-256.* The key exchange uses P-256, as most TLS clients still offer; X25519 cannot be used with an additive joint share.
5. *Traffic analysis.* N sees the size of every record. The body length is visible, and so is the response's token-length sequence, because streaming providers emit one record per token or small token group. This side channel has been shown to reconstruct a large share of LLM responses (Weiss et al., USENIX Security 2024). Compared with a node that sees the plaintext it is strictly less exposure; compared with "ciphertext only", it is more than volume and timing. The prompt side can be padded inside the JSON body; the response side needs provider-side padding or batching.
6. *DoS and limits.* The node and its relay validate the setup and reserve before opening any connection to U, bound concurrent sessions (32), the time to setup (15 s), the session lifetime (600 s from the node's WebSocket accept), the 2PC (180 s) and the body (8 MiB); before the credential record is written the relay buffers no more of C's bytes than the declared body needs, and C cannot make N preallocate more proof-channel correlations than N asks for itself. The 2PC's preprocessing makes a session expensive for N (about 22 MB of upload and 0.5 s of CPU each), which a reservation per session limits, but per-client limits must also bound it; the node refuses new sessions from a client address with twelve unused ones (connected but with no credential record written yet).
7. *Error metadata.* On an upstream error N learns the status line, the response headers and the body length, but not the body.
8. *Endpoint hygiene.* Keys live in C's process; N holds the API key in memory. C's certificate validation is RFC

5280 path validation to the WebPKI roots without revocation checks.

9. *Interactivity.* There is no artifact a third party can check. C holds $s_{ap,0}$ and could forge server records, so a portable artifact would need a design in which no single party holds the server key.
10. *Provider drift.* A provider change that breaks parameter pinning or usage reporting turns a verified session into an unverifiable one, which clients treat as failure, not fallback. The profile gate must be re-run per provider.
11. *Balances after an abort.* If N cuts the stream before the usage event, it charges the reservation, and C cannot prove who aborted. This is the trust the existing Routstr design already places in the node; in the honest case the cost is computed from upstream-authenticated usage, unbilled errors and sessions that never reached U cost nothing, and the receipt binds all of it. The amount at N's discretion is the reservation for the requested model.

6. Implementation and evaluation

6.1 Prototype

The implementation consists of:

- `cu-crypto`: native TLS 1.3 key schedule and GCM, tested against RFC 8448 and NIST vectors.
- `twopc`: the two-party key schedule (ECtF over P-256, the phase A and B circuits, a three-halves garbler and evaluator, IT-MACs on N's masks from `mpz2`'s Ferret and KOS correlated OT, the product and consistency checks), with a client-side binary.
- `zkbench`: the VOLE-ZK statements π_{NK}, π_{C2S}, π_{C3S} (and those of the single-party variant) with tests that the circuits agree with native code, mutation tests, and one-bit disclosure tests; the client's prover binary reads its job and witness on stdin, so no secret touches the disk.
- `cu-sidecar`: N's relay (Rust). It enforces the ClientHello and ServerHello rules, runs N's side of the 2PC, verifies the handshake fail-closed, assembles and writes the epoch-0 records, holds the final body record, and verifies C's proofs.
- A reference TypeScript client, and the production path: a transport in the Routstr client SDK used by the client daemon, and a WebSocket route in the node that relays to the sidecar and bills through the node's ordinary accounts.

C's TLS stack is a fork of an MIT-licensed TLS 1.3 client with two hooks: an external key share, and traffic secrets injected from the 2PC instead of computed locally. It accepts only the version, suite and group it offered, validates certificate paths per RFC 5280 and hostnames per RFC 6125, and uses the full 64-bit sequence number in the nonce.

6.2 Component costs

All component figures are from an Apple M2 Max, release builds, both parties in one process. "Local" has no delay. Network figures use an emulated link with a rate per direction and a fixed one-way delay.

Two-party key schedule.

Circuit	AND gates	XOR gates	SHA-256 compressions	Output bits
Phase A	298,088	1,421,372	14	768
Phase B	358,091	1,705,600	17	1,504

N garbles each circuit once, 656,179 AND gates per session. Only 8 of phase A's compressions are on the path to c_{hs} and s_{hs}.

Step (local, C's view)	Time	C sends / receives
Preprocess (tables, MACs, checks, OT setup, OLEs)	0.55 s in-process; 0.70-0.84 s in the live client	1.21 MB / 22.07 MB
Joint key share	2 ms	41 B / 74 B
ECtF	5 ms	0.2 kB / 12 kB

Step (local, C's view)	Time	C sends / receives
Phase A (evaluation and check)	33 ms (32-47 ms live)	73 B / 8 kB
Phase B (evaluation and check)	40 ms (38-40 ms live)	238 B / 24 kB

Of the 22 MB C downloads, 16.2 MB are the three-halves tables and 5.3 MB the 64-bit check rows; the rest is the mask commitments and the correlated OT. Of the 1.21 MB it uploads, 1.15 MB is its part of Ferret, in which C is the sender. Ferret is the only backend: KOS alone would upload almost nothing but would make N send 16 bytes per MAC, 42.7 MB in all, and with N scheduling its tables around C's upload Ferret was faster on every link we measured. N spends about 0.5 s of CPU per session (six hash calls per AND gate, Ferret, the check material) and C about 0.4 s; each online phase takes 35 to 45 ms.

Across a network the online steps take about 0.5 s at 80 ms RTT: the key share 84 ms, ECtF 170 ms, phase A about 121 ms and phase B about 126 ms, each phase about 1.5 round trips. Preprocessing is bandwidth-bound and dominates a cold session (section 6.4).

Proofs. The anchors remove the key schedule from the client's proofs, and π_{NK} replaces the single-party variant's pair $\pi_{C1} + \pi_N$ (section 7):

Proof	Local	50 ms RTT	Data	SHA-256 compr.	AES blocks
π_{NK} (credential record)	0.16 s	1.23 s	1.45 MB	5	15
π_{C2S} (suffix)	0.16 s	1.22 s	1.45 MB	7	15
π_{C3S} (usage)	0.20 s	1.27 s	1.47 MB	7	38
v1 π_{C1} (record material)	0.46 s	1.57 s	1.66 MB	32	18
v1 π_N (head template)	0.08 s	0.79 s	0.95 MB	4	0
v1 π_{C2} (suffix)	0.47 s	1.58 s	1.67 MB	36	15
v1 π_{C3} (usage)	0.46 s	1.56 s	1.67 MB	28	38

The 50 ms RTT column uses 100 Mbit/s. At 50 ms RTT about 0.7 s of each proof is fixed cost (OT setup and the final flush). Data sizes move in steps of about 0.48 MB per direction because the OT extension runs in fixed chunks.

6.3 End to end

Live sessions against api.venice.ai (`deepseek-v4-flash`, 40 tokens), client and node on one host, medians of 5 sessions each. The single-party variant (v1, section 7) is measured on the same setup.

	v1 (single-party)	v2 (this protocol)
Cold: request start → body sent	0.73 s	1.2 s (2PC preprocessing 0.76 s; π_{NK} 0.06 s); one run
Cold: suffix proof held before U starts	0.45 s (π_{C2})	0.10 s (π_{C2S})
Cold: time to first token	1.90 s	2.43 s †
Pre-warmed: arming time per session	0.71 s	0.88 s †
Pre-warmed: claim → body sent	3 ms	5 ms
Pre-warmed: time to first token	1.41 s	0.90 s †

† Measured with the dual-execution 2PC (section 6.6). Its local preprocessing took 0.49 s against 0.70-0.84 s now, so arming and the cold time to first token are now a few hundred milliseconds longer; its online phases took 27 and 33 ms with their checks, against 33 and 40 ms now, so the pre-warmed path is essentially unchanged.

On one host the protocol's own path to U starting work is 1.18 s for v1 and about 1.3 s for v2 cold, and 0.45 s against 0.10 s pre-warmed. The time-to-first-token figures are dominated by the provider's own latency, which varied by several hundred milliseconds during the runs. An earlier interleaved comparison of the single-party variant with a direct HTTPS call (60 tokens, n = 8) measured +1.07 s to first token cold and about +0.25 s pre-warmed. π_{C3S} and settlement run after the client's stream has ended and add no latency.

Pre-warming. Because the garbled tables are single-use and N may write only in epoch 0, the preprocessing cannot be shared between requests; pre-warming is how the protocol hides it. A client arms a session before a request exists: setup and reservation, preprocessing and the joint key share. It stops there, before the ClientHello, so an armed session holds no connection to the provider (N dials U only when the ClientHello arrives). A request claims the session: the handshake with both online 2PC phases, π_{NK}, the credential record and KeyUpdate, the body and π_{C2S}. An armed session holds a reservation until it is claimed or discarded (released at no charge), and its body length is fixed at arming: the client pads a shorter body with JSON whitespace between the private part and the suffix, choosing the length from recent bodies (power-of-two buckets). The client daemon arms only while it sees traffic: after a confidential request for a given node, model and token cap it keeps one armed session, two once that key has seen three requests in five minutes, and at least as many as the largest number of its requests that overlapped (at most six per key and six in total, and six in total for nodes that charge an abandon fee). It retires an armed session 480 s after its setup, before the node's session limit, and closes a key's sessions once it has had no request for ten minutes, so an idle client holds nothing. (The reference client's `--arm` mode, used for the local benchmarks, arms further, through the handshake and π_{NK}.)

The proofs' own setup is pre-warmed too. Each proof needs OT/VOLE correlations (Chou-Orlandi base OT, KOS, Ferret), and generating them per proof costs about twelve round trips and up to 1.5 MB, which over a home uplink took 5.4 s for π_{NK} and 3.0 s for π_{C2S}. These correlations are independent of the request, so an armed session also opens one proof channel and generates, during arming, the correlations for both directions (N proves π_{NK}, C proves π_{C2S} and π_{C3S}), with a fresh Δ per session; each correlation is used once, a failed proof closes the channel, and a session without a channel falls back to one connection per proof.

Against a deployed node. We measured against a deployed node in a datacenter, which serves at more than 90 Mbit/s within it. Over an earlier home connection (about 2-7 Mbit/s up), with the proof channel pre-warmed, π_{NK} took 0.21-0.25 s, π_{C2S} 0.04 s and π_{C3S} 0.06 s, against 5.4 s and 3.0 s for the first two with per-proof setup. With the dual-execution 2PC, a cold request from that connection reached its first token after about 35 s, almost all of it the client uploading its half of the preprocessing, and a session armed up to the key share gave a first token after 2.0-3.2 s, most of it the provider's own latency.

A second set of runs used a poor, download-limited link: a laptop on congested 2.4 GHz Wi-Fi (up to about 900 ms to the local router) behind a satellite connection, with only about 2-5 Mbit/s from the node. Node-only and dual-execution builds were run back to back in the same hour:

Same link, cold	Node-only (v2)	Dual execution
2PC preprocessing	42.5-62.6 s (4 runs)	57.8-62.2 s
Time to first token	45-68 s	66-69 s
Client sends / receives	1.21 / 22.1 MB	about 21 / 21 MB

On such a link both versions are bound by the 21-22 MB the client must receive, so they perform alike. Node-only garbling helps where the client's upload is the bottleneck, as on typical asymmetric home links (7.4 s against 85 s at 2/50 Mbit/s in section 6.4), and is about a second slower on fast symmetric links. The online phases on this link took 0.21-0.39 s (phase A) and 0.18-1.6 s (phase B), following the link's variable latency. With a fully armed node-only session (through the handshake and π_{NK}), claim to body took 58 ms and the first token came after 0.94 s, of which 0.89 s was the provider's own latency; arming took 42.8 s in the background. A cold request thus waits about a minute on this link, while a pre-warmed one waits about 1 s when fully armed, or 2-3 s with the SDK's pre-warming, which arms only up to the key share.

The same laptop on a good link (5 GHz Wi-Fi, about 100 Mbit/s down, 60-80 ms to the node) gives the numbers a typical user should expect from node-only v2:

Good home link	Node-only (v2)
Cold: 2PC preprocessing	5.1-11.7 s (3 runs)
Cold: online 2PC, proofs (over the channel), provider	0.3 s, 0.3 s, 1.3 s
Cold: time to first token	13.8-16.2 s
Fully armed: arming (background) / claim to body / first token	6.7 s / 73 ms / 1.05 s

The cold path is still dominated by receiving the 22 MB of garbled tables; pre-warming removes it from the request path.

6.4 Over a network and bandwidth

A cold 2PC session (preprocessing, key share, ECtF and both phases, until both parties finish) on an emulated link, median of 3 runs, for the earlier dual-execution version (section 6.6), for the first node-only implementation (half-gates tables, 128-bit check rows) and for the current one:

Link (one-way delay; up / down for the client)	Dual execution	Node-only, first version	Node-only (v2)
local	0.17 s	0.67 s	0.63 s
40 ms; 2 / 50 Mbit/s	85.2 s	11.6 s	7.38 s
40 ms; 7 / 50 Mbit/s	24.9 s	8.27 s	5.41 s
25 ms; 100 Mbit/s symmetric	2.27 s	4.07 s	3.05 s
40 ms; 2 / 10 Mbit/s	85.2 s	32.2 s	20.0 s
Client sends / receives	21.1 / 21.1 MB	1.21 / 32.2 MB	1.21 / 22.1 MB

Dual execution on the 2 and 7 Mbit/s uplinks is a single run. Node-only garbling turns the client's 21 MB upload into a 1.2 MB upload and a 22 MB download, which suits asymmetric home links: on a 2 Mbit/s uplink the cold 2PC drops from 85 s to 7.4 s. The floor there is C's Ferret upload, about 4.9 s; N holds its tables back until C's upload is under way, so that they do not delay the small messages that Ferret's round trips wait for. On a fast symmetric link node-only garbling is slower than dual execution (3.05 s against 2.27 s), because 22 MB travel one way instead of 21 MB each way in parallel and the checks add sequential steps. With a 10 Mbit/s downlink the download dominates (20 s). The online phases are faster than with dual execution, because there is no separate equality-check round trip: at 40 ms one way, phase A takes about 121 ms against 207 ms and phase B about 126 ms against 287 ms.

The critical-path work before U starts, with client and node 50 ms apart at 100 Mbit/s (sums of the component benchmarks with per-proof setup, without the relayed handshake round trips that both versions share):

	v1	v2
Cold	3.9 s ($\pi_{C1} + \pi_N + \pi_{C2}$)	about 5.5 s (2PC 3.05, π_{NK} 1.23, π_{C2S} 1.22)
Pre-warmed	1.6 s (π_{C2})	1.2 s (π_{C2S}); 0.04 s over a pre-warmed proof channel

Cold, v2 is somewhat slower than v1 on a fast link and several seconds slower on a slow uplink (the 2PC alone takes 7.4 s at 2 Mbit/s). Pre-warmed, v2 is faster, because only its cheaper suffix proof is on the path.

Data per request: v1 moves about 5.9 MB of proofs; v2 about 23.3 MB of 2PC traffic (22.1 MB to the client, 1.2 MB from it) and 4.4 MB of proofs, about 28 MB in all. N uploads about 24 MB per v2 request (100 requests per minute is about 0.3 Gbit/s), against about 3 MB for v1, and spends about 0.5 s of CPU on the 2PC. The 2PC cost is constant in the size of the prompt and the response.

6.5 Live verification

Live sessions against the case-study provider completed with all proofs verified, a signed receipt passing every client check, and the cost within the reservation (63 of 2,282 msat in the example of section 4.6). They were billed through the node's ordinary accounts, with both an API key and a one-off e-cash token as the credential. Sessions over the production SDK transport and the client daemon, driven by an off-the-shelf OpenAI client, settled; the node logged no plain chat request. Earlier runs of the same settlement path covered an upstream 400 settled at zero cost with only the response head disclosed (the provider's error body quoted the request's private field name, and no log contained it), and idle connections that never sent a setup, which caused no connection to the provider.

Adversarial runs form a regression gate (`crypto/tools/adversarial-model-swap.sh`) with an honest control and an expected outcome per case, checked against the node's billing log. On v2 it passed 6 of 6: the control settled, a decoy model in the private part of the body settled as the setup model, and every attack was refused with the reservation released:

- a suffix other than the canonical one;
- a body naming a pricier model, or a model the node does not offer, with the suffix withheld (U never received a complete request);
- a body ending in another model's suffix while claiming the canonical one (π_{C2S} fails).

A client that puts its own P-256 key share in the ClientHello instead of the joint one is refused before anything reaches the provider. The unit and integration suites of all components pass.

6.6 The dual-execution version

The first implementation of v2 used dual execution (Mohassel and Franklin 2006; Huang, Katz and Evans 2012) instead of node-only garbling. Each phase was garbled twice with `mpz2`'s half-gates garbler: N garbled and C evaluated, and C garbled and N evaluated, both runs at once. Every output was decoded by both evaluators, and before using any output the parties compared the two runs through a commit-then-open hash of output labels, which a party can produce for a differing bit only with an inactive label of the other's circuit. Nothing was ever opened. Per phase the adversary learned the outcome of the comparison, $\log_2 3 \approx 1.6$ bits per session about fresh values, under the same invariant as now.

It worked and was deployed, but each party had to garble 656,179 AND gates and send the other about 21 MB of tables. The client's upload made a cold session take 25 s on a 7 Mbit/s uplink and 85 s on a 2 Mbit/s one (section 6.4), about 35 s to the first token from a home connection, and a minute of background upload for every pre-warmed session. Node-only garbling removes that upload. Dual execution remains a sound alternative: it is simpler, it rests on better-studied constructions (half-gates garbling and the dual-execution equality check), it leaks less (about 1.6 bits per session against about 2.3), it is faster on fast symmetric links, and it needs no Ferret. It is kept as the tag `v2-symmetric` in the three repositories (routstr-confidential `0876b5e`, routstr-core `dd9e51a5`, routstr-sdk `b5d16f0`).

7. v1: the single-party variant

The protocol has a simpler variant, which we call v1, without any 2PC. A node runs one version and announces it in its signed offer; clients follow the offer.

How it works. C runs the TLS handshake alone, with an ordinary X25519 key share, and discloses only c_{hs} and s_{hs} to N, which verifies the handshake as in v2. C then sends a KeyUpdate in epoch 0, so that epoch 1 starts. For epoch-1 sequence 0, C gives N **one-shot record material**: the CTR keystream truncated to the head length, the GHASH key and the tag mask. π_{C1} proves to N, with HS as the witness, that this material comes from the session's key schedule (HS through MS, $c_{ap,0}$, $c_{ap,1}$, K_1, IV_1), so that C cannot hand out a keystream that rewrites the head around N's key. N forms its record by XOR, proves its content with π_N (the v2 π_{NK} with a public keystream), and writes it; C then sends a second KeyUpdate and writes the body in epoch 2. π_{C2} and π_{C3} prove the suffix and the usage event like their v2 counterparts, but take HS as the witness and anchor on the public handshake values, so each recomputes the key schedule (28 to 36 compressions).

What it gives up. C holds the whole epoch-1 key K_1: it needs it to seal its own KeyUpdate at sequence 1. The API key is therefore exactly $C_1 \oplus S$, and P1 holds only as long as C never sees C_1. The implementation never sends it to C, but anyone on N's egress path who colludes with C (a shared cloud network, a TLS-inspecting middlebox, the hosting provider, a CDN edge) can recover the key, and one leak exposes it until it is rotated. Short-lived keys with spend caps limit the loss without removing the assumption. In v1 P1 is an operational assumption; in v2 it is a cryptographic property. P2 to P6 are the same in both versions, and v1's residual risk 1 is narrower: N can write exactly one record of fixed length.

When v1 is preferable.

	v1	v2
Client can recover the API key	yes, if it sees N's egress ciphertext	no
New cryptography in the trust base	ZK proofs	ZK proofs and a 2PC
Key exchange	X25519	P-256
Data per request	≈5.9 MB	≈28 MB (22 MB of it to the client)
Node upload per request	≈3 MB	≈24 MB
Cold critical-path work	≈3.9 s (50 ms RTT, 100 Mbit/s)	≈5.5 s (50 ms RTT, 100 Mbit/s); 2PC alone ≈7.4 s on a 2/50 Mbit/s link
Pre-warmed suffix proof	0.47 s local, 1.6 s at 50 ms RTT	0.16 s local, 1.2 s at 50 ms RTT

v1 suits nodes whose upstream path is under their own control, clients on slow or metered links (above all slow downlinks), and clients that cannot pre-warm. v2 suits nodes on shared or untrusted infrastructure and nodes with valuable keys, and is faster when sessions are pre-warmed.

8. Deployment pattern

Confidential mode plugs into an existing routing deployment as an alternative *transport*; everything else stays the deployment's own code. Negotiation is per request and stateless:

1. **Offer.** The node publishes a signed offer (protocol version, upstream hostname, key length and alphabet, head template, pinned suffix keys, a signed rate snapshot, attestation key) at a public endpoint.
2. **Model listing.** The model listing gains one additive field per model that references the offer; old clients ignore it.
3. **Opt-in.** A client opts in per request, for `POST /v1/chat/completions` only. It routes only to nodes that advertise the field for the model, and requires that the offer signature verifies with a pinned node key (not just the key the offer names), that the upstream hostname is on its trusted hostname list (an empty list refuses every host), and that its prover is available. It rebuilds every public input of π_{NK} from the signed offer.
4. **Billing** is the node's ordinary billing (section 4.8). The client's credential, reservation, settlement, balance and refunds are the existing ones. In the reference integration the node adds a WebSocket route and an offer; the client SDK adds one transport branch beside its other transports, so credential acquisition, top-ups, failover and balance accounting are unchanged; the daemon adds a configuration block and passes the opt-in through.
5. **Response.** The client streams the decrypted body to the application as records arrive, or folds the stream into one completion object if the caller did not ask for streaming (the pinned suffix always requests streaming, because the usage event only exists there). Checks that complete after the response (the disclosure proof, the receipt, the usage, the cost, the served model) are not dropped: a failure fails a non-streaming call and ends a stream with an error.

For existing deployments the change is additive: the ordinary proxy path and old clients are untouched. Requests that opted in fail closed and are never sent on the plain path, since a silent downgrade would break the privacy promise.

8.1 Denial-of-service protection and fees

A confidential session is expensive for the node before the provider sees a byte: N garbles the key-schedule circuit and sends about 22 MB, spends about 0.5 s of CPU, and holds a reservation on the client's credential. Pre-warming makes

this cheaper to abuse, because a session can be armed without ever being used. We therefore layer several limits, from free and always on to priced and optional.

1. **Nothing reaches the provider before a request.** The relay dials U only when C's ClientHello arrives, after setup and the 2PC preprocessing, so armed and abandoned sessions never open provider connections and the provider sees exactly one connection per completed or attempted request.
2. **Validation before work.** N validates the setup and the credential and makes the reservation before it contacts the relay; the relay validates the setup before it starts the 2PC. A client without funds gets no garbled circuit.
3. **Global and per-address caps.** N serves at most a fixed number of concurrent sessions (32 by default), each limited to 600 s from the node's WebSocket accept, with 15 s to send its setup. (The relay has its own 600 s limits, one for an armed session to send its ClientHello and a fresh one after it, so in practice the node's limit is the one that ends a session.) N also refuses a new session from a client address that already holds a set number of unused sessions, counted from the WebSocket accept, before the setup arrives, until the credential record is written (12 by default; HTTP 429). The address is the TCP peer, or, only for peers configured as trusted reverse proxies, the rightmost `X-Forwarded-For` entry; a client cannot name its own address. Clients behind one address, such as all Tor users of an onion service, share the limit.
4. **Fees in the signed offer (section 4.8).** A session fee f_s on completed requests, an abandon fee $f_a \geq f_s$ on sessions that end without a completed request, and a rate surcharge, all 0 by default. The abandon fee works as a deposit: it is reserved at setup and, when the session is used, all but f_s of it is in effect returned. A completed request therefore never pays f_a, while every abandoned session costs its holder f_a. C must echo the fees it accepts, so a client that does not know about them is refused rather than charged.
5. **Client-side restraint.** The client daemon arms sessions only while it sees confidential traffic, keeps a bounded number of armed sessions for a node that charges an abandon fee (six by default, sized for parallel coding agents), arms only if that fee is below a configured limit (5 sats by default), retires an armed session 480 s after its setup, and closes a key's armed sessions once it has had no request for ten minutes; it logs every abandon fee it pays.

How the deployed node is set up. The case-study node charges no session fee and no surcharge, and an abandon fee of 2 sats. A typical confidential chat on its cheapest models costs 0.1 to 1 sat of inference (25 msat for a short exchange in our test), so the fee would dominate the price of a completed request if it were charged there; instead only waste is priced. A paying user pays exactly the inference; an idle pre-warmed session costs 2 sats; and a client opening sessions it does not use is limited to twelve at a time per address, each costing 2 sats. (The limit was three at first; parallel coding agents, whose subagents each open a session per turn, ran into it, and every session is already backed by a funded worst-case reservation, so money rather than the count is the main brake.) We measured both cases on the live node: a completed request was charged 25 msat, an abandoned session 2000 msat. Separately, each request reserves the model's worst-case prompt for its duration (about 157 sats for a model with a 1M-token context at these rates) and releases what it does not use, so a credential needs that much balance to start a session even though it is not spent.

What remains. The fees and limits bound the cost per session and per address, not the total: many addresses can still open sessions, each paying f_a, which is a priced service rather than an attack. An operator who wants a different trade-off raises f_a, lowers the per-address limit, or charges a session fee to cover the node's bandwidth.

9. Related work and alternatives

MPC-TLS (TLSNotary). The prover and a notary jointly act as the TLS client, and all record encryption and decryption runs inside the MPC. With the roles arranged so that the node is the notary, it gives prompt confidentiality and lets the API key be the node's private input, and it works with TLS 1.2. The cost scales with the payload and there is no streaming: we measured about 5.8 s and 41.5 MB for a 2 KB request, 34 s and 377 MB for 32 KB, and 129 s and 1.44 GB for 128 KB (100 Mbit/s, 50 ms). Our 2PC moves about half the data of the 2 KB case, and it covers only the key schedule, so its cost is constant, can be precomputed, and the payload then streams at native speed. TLSNotary's key-exchange conversion is the basis of our ECtF.

DEAP and Garble-then-Prove. TLSNotary's dual execution with asymmetric privacy (DEAP) and Garble-then-Prove (Xie et al., USENIX Security 2024) obtain malicious security cheaply by having one party open its inputs, or its key shares, after the session, so that the other can check the garbling and later prove statements about the transcript.

That fits a notary whose shares are worthless after the session. It does not fit our setting: N's share would reveal the master secret, hence K_0, hence the API key, which is a long-term secret. We need a 2PC in which nothing is ever opened. Our first version used full dual execution with an equality check on output labels (Huang, Katz and Evans 2012; section 6.6); the current one uses garbler-only authenticated garbling.

Authenticated garbling (Wang, Ranellucci and Katz 2017; Katz, Ranellucci, Rosulek and Wang 2018; Dittmer, Ishai, Lu and Ostrovsky 2022; Cui, Wang, Yang and Yu 2023) makes one garbled circuit secure against a malicious garbler. The wire masks are secret-shared between the parties and the AND gates are backed by authenticated triples, so nothing leaks. KRRW-style triple generation costs more bandwidth, in both directions, than garbling twice, and the compressed preprocessing of DILO and CWYY needs primitives (block correlated OT or subfield VOLE with many keys) that `mpz2` does not provide. We keep the masks with the garbler and authenticate them to the evaluator. That needs only correlated OT and a QuickSilver-style product check, and its batched per-wire check follows CWYY's, but it accepts a selective-failure leakage of at most 2.3 bits per session. Its soundness argument is our own (section 5). Three-halves garbling (Rosulek and Roy 2021) cuts N's tables by about a quarter against half-gates.

DiStefano (Celi et al., 2023) runs the TLS 1.3 handshake as a 2PC with fully authenticated garbling, which is maliciously secure without leakage but needs heavier preprocessing. It could replace our 2PC where the remaining bits of leakage matter more than bandwidth.

DECO (Zhang et al., ACM CCS 2020) introduced the three-party handshake, in which prover and verifier secret-share the session keys so that the verifier is convinced of data the prover received. Its goal is to convince the verifier of provenance; ours is the reverse split, in which each party receives a different epoch's key in full and the payload never enters the MPC.

Proxy zkTLS attestors (the "verified mode" deployed on Routstr nodes; Reclaim-style attestors). The node is the TLS client and an attestor checks its transcript. This gives request and response integrity and portable proofs, but the node and the attestor see the plaintext; we measured +10.8 s to first token on a live node.

TEE (enclave) designs. The node runs in a hardware enclave that holds the API key and processes the prompt in the clear, and users verify attestations. This is fast and streams, and it works with any provider. It rests on the hardware vendor and on the measured code, and the guarantee is only as good as the user's review of every software version the enclave runs (section 1).

The two approaches compose. Our protocol removes the need to trust the reseller in the middle, while C remains the TLS endpoint of the provider. If the provider itself serves inference from an enclave and binds its TLS key or an encryption key to an attestation (attested or end-to-end-encrypted inference offered by some providers), C can verify that attestation end to end through N, which only relays sealed records. The user then gets math-based guarantees about the node and the provider's attestation about the provider, without having to review the node's software.

	This work (v2)	v1 variant	MPC-TLS	Proxy zkTLS	TEE
Prompt hidden from node	yes	yes	yes	no	yes
API key hidden from client	yes	if N's egress is private	yes	yes	yes
Trusted hardware or attested software	no	no	no	no	yes
Streaming, native-speed payload	yes	yes	no	yes	yes
Added latency, pre-warmed (local)	0.10 s proof	0.45 s proof	5.8-129 s	≈10 s	≈0
Data per request	≈28 MB, constant	≈5.9 MB	41 MB-1.4 GB	transcript size	≈0

	This work (v2)	v1 variant	MPC-TLS	Proxy zkTLS	TEE
Third-party-verifiable transcript	no	no	yes	yes	attestation only
Provider requirement	TLS 1.3 profile with P-256	TLS 1.3 profile	TLS 1.2+	TLS 1.2/1.3	any

The approaches compose: proxy zkTLS provides integrity for unmodified providers, MPC-TLS covers providers that fail the profile, and our design is the fast lane where a provider's TLS and JSON semantics cooperate.

10. Conclusion

We presented a protocol that gives LLM routing users prompt confidentiality, request and response integrity, and upstream-anchored accounting against a node that relays ciphertext, and gives the node a guarantee that its API key stays hidden from the client, with no trusted hardware and no MPC over the payload. Its trust rests on the provider's TLS certificate and on cryptography, not on attested software. The mechanisms are a two-party TLS key schedule that gives the node the first application epoch and the client every other one, a proven public suffix for parameter pinning, and a proven usage disclosure for settlement. A complete implementation, verified against a live provider, reaches the provider 5 ms after a pre-warmed request is claimed, plus a 0.10 s suffix proof. Its cost is about 22 MB of precomputable tables and check data sent to the client and 1.2 MB sent back per request, because only the node garbles and the client checks the garbling instead of garbling a second copy; a cold two-party computation takes about 7 s on a home link with a 2 Mbit/s uplink. The price of that asymmetry is a selective-failure leakage of at most 2.3 bits per session about values that are never reused. The single-party variant needs under a quarter of the bandwidth where the node can keep its egress private.

Appendix A. Case study: the provider profile in practice

A.1 Verified behaviours (live gate `crypto/tools/provider-profile-check.py`, checks A1-A11, 11/11 on 2026-10-10; rate limits, 5xx and timeouts count as inconclusive, never as a pass, and each rejection check asserts the exact status and error text against a valid control).

- **Transport:** a client offering only TLS 1.3 with `TLS_AES_128_GCM_SHA256` is served; a client KeyUpdate sent in the middle of the request is accepted and the server answers completely and closes (`Connection: close`, HTTP/1.1). The v2 live sessions establish that a single secp256r1 key share is accepted without a HelloRetryRequest.
- **Waits for the full body:** nothing for 8 s with the body incomplete, then a response within 0.7 s of the last bytes.
- **Last-wins:** a bad model then a good one gives 200; a good one then a bad one gives 404; a `max_tokens` of 200 followed by 8 behaves like 8.
- **Replace, not merge:** a later `venice_parameters` object replaces an earlier one; an invalid `character_slug` that is overridden is ignored.
- **Strict parsing (sampled):** an unknown top-level field and malformed JSON are rejected with 400, including a suffix landing in an open object or string. The 400 body for an unknown field quotes the field's name, which is why an error disclosure covers only the response head (section 4.8).
- **Token cap:** `max_completion_tokens` caps output including reasoning, with a small overshoot on tiny caps (3 billed up to 6, 40 billed 39). The reservation covers this easily.
- **Input bound:** with the pinned `include_venice_system_prompt: false` a 224-byte request billed 6 prompt tokens. Without the pin the provider injects about 1.6k tokens of system prompt.
- **Unpinned price fields (sampled):** `service_tier`, `web_search_options` and `prediction` are rejected.
- **Images:** every offered model rejects image parts, both `image_url` and data URIs (`crypto/tools/regression-modality.py`, 8/8).

- **Usage event:** the terminal event carries usage, cost and model with empty `choices`, followed by `[DONE]`.
- **Fake usage events:** a prompt instructing the model to emit them produced none. Every imitation was escaped inside a JSON string, and the raw stream contained no CR/LF inside events.

A.2 Profile application. The pinned suffix for this provider: `,"model":M,"max_tokens":T,"max_completion_tokens":T,"n":1,"` with `max_completion_tokens` pinned because it overrides `max_tokens` (a bypass observed in testing) and the provider-specific parameters pinned to keep web search and system-prompt injection off the bill.

A.3 Other providers. The generic pieces (epochs, 2PC, proofs) port unchanged. The open questions per provider are profile items 4 to 6 and 8, plus whether HTTP/1.1, P-256 and a mid-request KeyUpdate are accepted (some CDNs pin HTTP/2). Where a provider fails, the fallbacks are MPC-TLS for confidentiality or the existing integrity-only path. Per-provider assessments are in the project's `docs/ADVERSARIAL-QA.md`.

References

- S. Celi, A. Davidson, H. Haddadi, G. Pestana, J. Rowell. *DiStefano: Decentralized infrastructure for sharing trusted encrypted facts and nothing more.* NDSS 2025 (ePrint 2023/1063).
- H. Cui, X. Wang, K. Yang, Y. Yu. *Actively secure half-gates with minimum overhead under duplex networks.* EUROCRYPT 2023.
- S. Dittmer, Y. Ishai, S. Lu, R. Ostrovsky. *Authenticated garbling from simple correlations.* CRYPTO 2022.
- Y. Huang, J. Katz, D. Evans. *Quid-Pro-Quo-tocols: Strengthening semi-honest protocols with dual execution.* IEEE S&P 2012.
- J. Katz, S. Ranellucci, M. Rosulek, X. Wang. *Optimizing authenticated garbling for faster secure two-party computation.* CRYPTO 2018.
- M. Keller, E. Orsini, P. Scholl. *Actively secure OT extension with optimal overhead* (KOS). CRYPTO 2015.
- P. Mohassel, M. Franklin. *Efficiency tradeoffs for malicious two-party computation.* PKC 2006.
- E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3.* RFC 8446, 2018; test vectors RFC 8448.
- M. Rosulek, L. Roy. *Three halves make a whole? Beating the half-gates lower bound for garbled circuits.* CRYPTO 2021.
- TLSNotary project. *mpz* and *mpz2* (garbling, oblivious transfer, VOLE-based ZK) and the MPC-TLS protocol documentation.
- X. Wang, S. Ranellucci, J. Katz. *Authenticated garbling and efficient maliciously secure two-party computation.* ACM CCS 2017.
- R. Weiss, D. Ayzenshteyn, G. Amit, Y. Mirsky. *What was your prompt? A remote keylogging attack on AI assistants.* USENIX Security 2024.
- X. Xie, K. Yang, X. Wang, Y. Yu. *Lightweight authentication of web data via garble-then-prove.* USENIX Security 2024.
- K. Yang, C. Weng, X. Lan, J. Zhang, X. Wang. *Ferret: Fast extension for correlated OT with small communication.* ACM CCS 2020.
- K. Yang, P. Sarkar, C. Weng, X. Wang. *QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field.* ACM CCS 2021.
- S. Zahur, M. Rosulek, D. Evans. *Two halves make a whole* (half-gates). EUROCRYPT 2015.
- F. Zhang, D. Maram, H. Malvai, S. Goldfeder, A. Juels. *DECO: Liberating web data using decentralized oracles for TLS.* ACM CCS 2020.